

智能识别系统设计



群名称:智能识别系统设计2023
群 号:261010657

上课地点: 深大南区致腾楼323

深圳大学
计算机与软件学院

● 参考资料

1. 数字图像处理（第三版） Rafael C. Gonzalez, Richard E. Woods著，阮秋琦，阮宇智译 电子工业出版社，2011. 6.
2. 模式分类（第二版）[美] 理查德·O. 杜达（Richard, O., Duda） 著，李宏东 姚天翔 等 译，2022. 5.

其他参考资料：

1. <https://opencv.org/>
2. OpenCV 3计算机视觉：Python语言实现，JoeMinichino著，机械工业出版社， 2016. 6.
3. 深度学习入门 基于Python的理论与实现 斋藤康毅著 人民邮电出版社，2018. 7.

- OpenCV中关于图像操作

基本数据结构:

`cv::Mat` //矩阵

`cv::Point` //点坐标 例: `cv::Point3d p( x0, x1, x2 );`

`cv::Scalar` //值向量 例: `cv::Scalar red( 0, 0, 255);`

`cv::Size` //大小向量 例: `cv::Size sz( w, h );`

`cv::Rect` //矩形向量 例: `cv::Rect rt( x, y, w, h );`

`cv::vec_` //向量 例: `Vec2f v2f(x0,x1);`

- OpenCV中关于图像操作

图像基本信息

`img_gray.rows;` // height of image

`img_gray.cols;` // width of image

`img_gray.channels();` // number of channels

图像像素

`img_src.at<uchar>(y, x);` //gray image

`img_src.at<cv::Vec3b>(y, x);` //color image

图像区域（ROI）

`img_rgb(cv::Rect(105, 18, 50, 50));` // clip image

`img_roi(roi_rect).setTo(cv::Scalar(255, 255, 255));` //roi



- OpenCV中关于图像操作

基本运算

最大，最小，均值，方差
求和，非零数
加、减、乘、点乘
等其矩阵操作



```
cv::addWeighted( roi1, alpha, roi2, beta, 0.0, roi1 );
```


- OpenCV中关于图像操作

图像缩放

`void cv::resize (InputArray src, OutputArray dst, Size dsize, double fx = 0, double fy = 0, int interpolation =INTER_LINEAR)`

图像镜像

`void cv::flip(InputArray src, OutputArray dst, int flipCode)`

$$\text{dst}_{ij} = \begin{cases} \text{src}_{\text{src.rows}-i-1,j} & \text{if } \text{flipCode} = 0 \\ \text{src}_{i,\text{src.cols}-j-1} & \text{if } \text{flipCode} > 0 \\ \text{src}_{\text{src.rows}-i-1,\text{src.cols}-j-1} & \text{if } \text{flipCode} < 0 \end{cases}$$

- OpenCV中关于图像操作

仿射变换（目标校正）

`cv::warpAffine()`

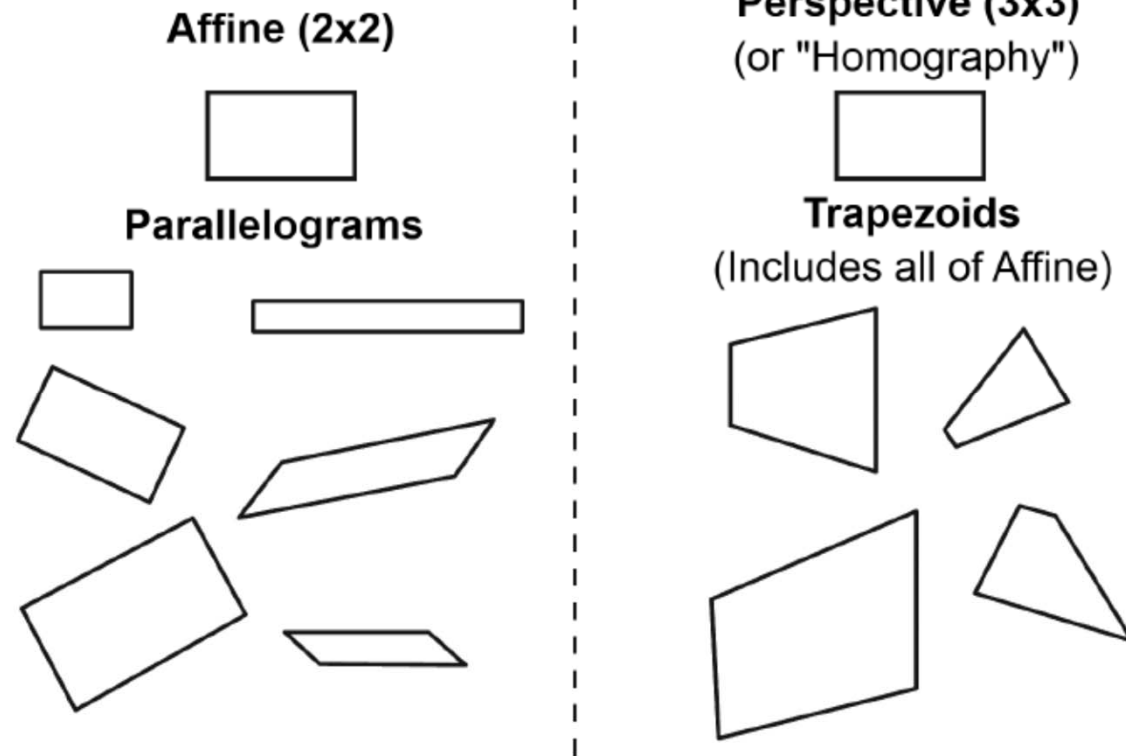
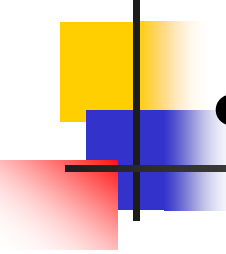


Figure 11-3. Affine and perspective transformations



● OpenCV中关于图像操作

仿射变换

```
void cv::warpAffine ( InputArray src, OutputArray dst, InputArray M, Size  
dsize, int flags = INTER_LINEAR, int borderMode = BORDER_CONSTANT, const  
Scalar & borderValue = Scalar() )
```

旋转矩阵

```
Mat cv::getRotationMatrix2D (Point2f center, double angle, double scale)
```

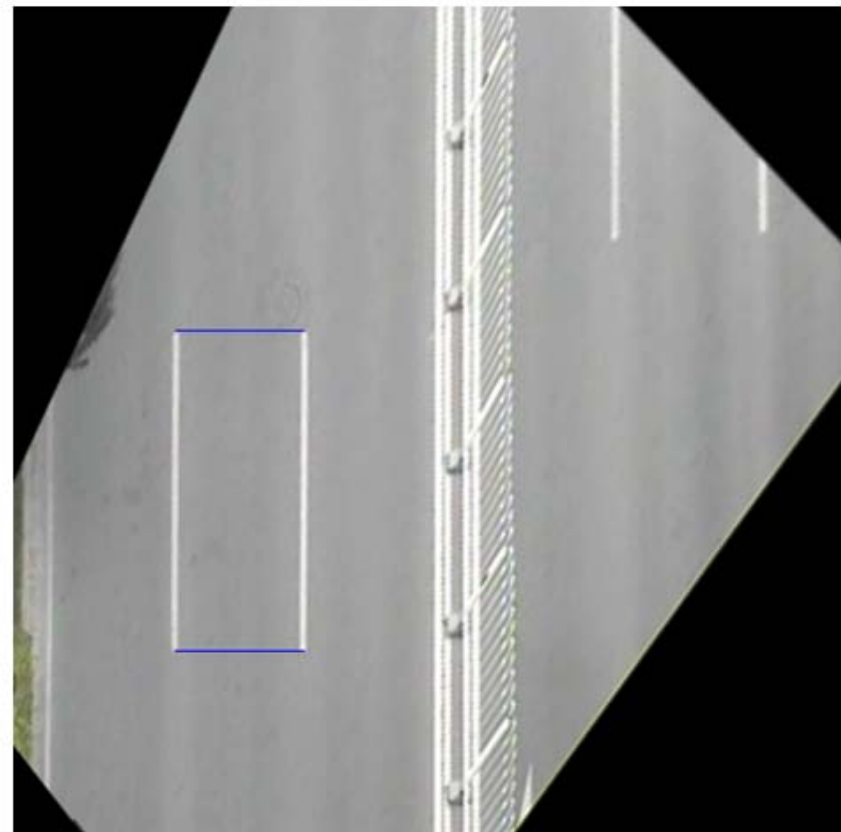
Ex1:

```
cv::Mat rot_mat = cv::getRotationMatrix2D(center, rotate_degree, 1.0);  
cv::warpAffine(img_src, img_rotate, rot_mat, src_sz);
```

Ex2:

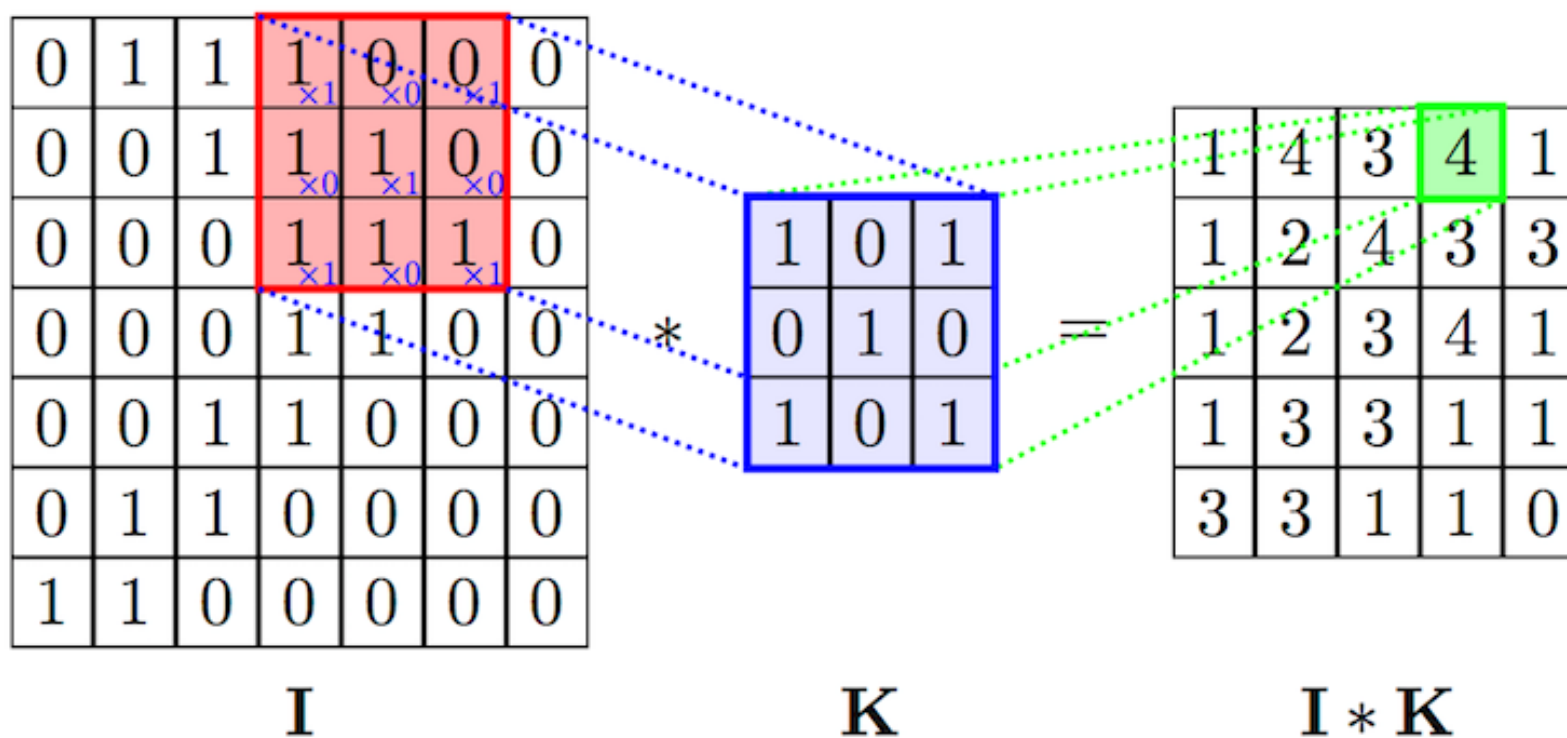
```
cv::Rect2f bbox = cv::RotatedRect(cv::Point2f(), img_src.size(),  
rotate_degree).boundingRect();  
// adjust transformation matrix--translation transformation  
rot_mat.at<double>(0, 2) += bbox.width / 2.0 - img_src.cols / 2.0;  
rot_mat.at<double>(1, 2) += bbox.height / 2.0 - img_src.rows / 2.0;  
cv::warpAffine(img_src, img_urotate, rot_mat, bbox.size());
```


- OpenCV中关于图像操作



- OpenCV中关于图像操作

空间卷积及卷积核（局部操作）



卷积核：大小，值

- OpenCV中关于图像操作

空间卷积滤波器（去噪）

均值滤波

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

```
void cv::blur(InputArray src, OutputArray dst, Size ksize, Point anchor =  
Point(-1,-1), int borderType = BORDER_DEFAULT)
```

中值滤波

```
void cv::medianBlur(InputArray src, OutputArray dst, int ksize)
```

高斯滤波

```
void cv::GaussianBlur(InputArray src, OutputArray dst, Size ksize,  
double sigmaX, double sigmaY = 0, int borderType = BORDER_DEFAULT)
```

双边滤波

```
void cv::bilateralFilter(InputArray src, OutputArray dst, int d, double sigmaColor,  
double sigmaSpace, int borderType = BORDER_DEFAULT)
```

- OpenCV中关于图像操作

形态学滤波器 (OCR, 指纹识别)

```
void cv::morphologyEx(InputArray src, OutputArray dst, int op,  
InputArray kernel, Point anchor = Point(-1,-1), int iterations = 1, int  
borderType = BORDER_CONSTANT, const Scalar & borderValue =  
morphologyDefaultBorderValue() )
```

```
enum cv::MorphTypes {  
    cv::MORPH_ERODE = 0,  
    cv::MORPH_DILATE = 1,  
    cv::MORPH_OPEN = 2,  
    cv::MORPH_CLOSE = 3,  
    cv::MORPH_GRADIENT = 4,  
    cv::MORPH_TOPHAT = 5,  
    cv::MORPH_BLACKHAT = 6,  
    cv::MORPH_HITMISS = 7  
}
```

```
enum cv::MorphShapes {  
    cv::MORPH_RECT = 0,  
    cv::MORPH_CROSS = 1,  
    cv::MORPH_ELLIPSE = 2  
}
```


- 本次课程任务（熟悉C++和Python两种编程环境）

1. 读取并显示图像
2. 遍历图像上所有像素
3. 操纵图像局部感兴趣区域
4. 滤波算子对图像进行滤波
5. 最优的梯度幅值图像
6. 学习对图像进行二值化的方法（动态阈值法）
7. 获取图像中的轮廓，对图像中的目标进行计数

本课程需要掌握的编程方法：

- 1) 类+结构体的框架进行编程；
- 2) STL模板库。

1. 读取并显示图像

```
#include "stdafx.h"
#include <opencv2/opencv.hpp>
using namespace std;
using namespace cv;

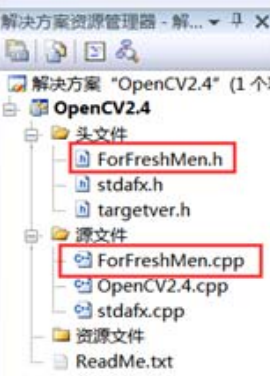
void Read_Show();

int _tmain(int argc, _TCHAR* argv[])
{
    Read_Show();
    return 0;
}

void Read_Show()
{
    const char* imagename = "boldt.jpg";
    //从文件中读入图像
    Mat img = imread(imagename);
    //如果读入图像失败
    if(img.empty())
    {
        fprintf(stderr, "Can not load image %s\n", imagename);
        exit(0);
    }
    //显示图像
    namedWindow("image", 1);
    cout<<"函数功能：读入并显示和保存一张图像"<<endl;
    imwrite("save.jpg", img);
    imshow("image", img);
    //此函数等待按键，按键盘任意键就返回
    waitKey(0);
}
```



类+结构体的框架进行编程



```

// OpenCV2.4.cpp: 定义控制台应用程序
//

#include "stdafx.h"
#include <opencv2/opencv.hpp>
#include "ForFreshMen.h"

using namespace std;
using namespace cv;

int main(int argc, char* argv[])
{
    CForFreshMen VarDemo;

    //函数功能：读入并显示和保存一张图像
    VarDemo.Read_Show();

    return 0;
}
  
```

```

#ifndef _FORFRESHMEN_H_
#define _FORFRESHMEN_H_
#include <opencv2/opencv.hpp>
#include <string>

using namespace std;
using namespace cv;

class CForFreshMen
{
public:
    CForFreshMen(void);
    ~CForFreshMen(void);

    void Read_Show();
};
#endif

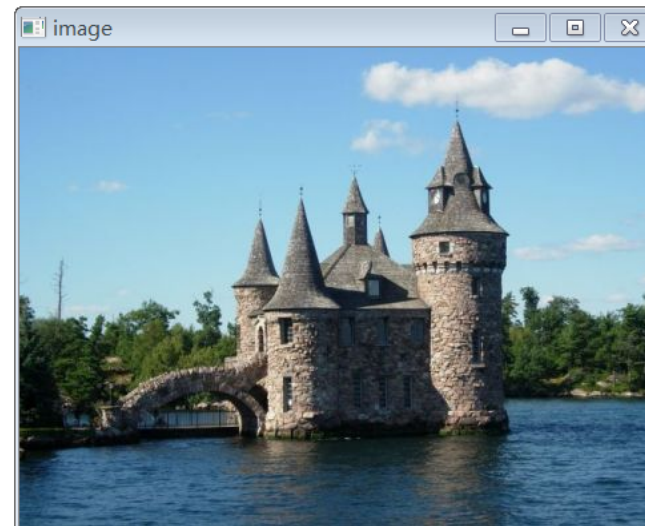
#include "ForFreshMen.h"

CForFreshMen::CForFreshMen()
{
}

CForFreshMen::~CForFreshMen()
{
}
  
```

类+结构体的框架进行编程

```
void CForFreshMen::Read_Show()
{
    const char* imagename = "boldt.jpg";
    //从文件中读入图像
    Mat img = imread(imagename);
    //如果读入图像失败
    if(img.empty())
    {
        fprintf(stderr, "Can not load image %s\n", imagename);
        exit(0);
    }
    //显示图像
    namedWindow("image",1);
    cout<<"函数功能：读入并显示和保存一张图像"<<endl;
    imwrite("save.jpg",img);
    imshow("image", img);
    //此函数等待按键，按键盘任意键就返回
    waitKey(0);
}
```



类+结构体的框架进行编程

International Journal of Computer Vision, 321-331 (1988)
© 1987 Kluwer Academic Publishers, Boston, Manufactured in The Netherlands

Snakes: Active Contour Models

MICHAEL KASS, ANDREW WITKIN, and DEMETRI TERZOPOULOS
Schlumberger Palo Alto Research, 3340 Hillview Ave., Palo Alto, CA 94304

SnakeModelTest - Microsoft Visual Studio(管理员)

文件(F) 编辑(E) 视图(V) VAssistX 项目(P) 生成(B) 调试(D) 工具(T) 窗口(W) 帮助(H)

解决方案资源管理器 - S... X

解决方案 "SnakeModelTest"

SnakeModelTest

头文件

- BasicSnakeModel.h
- MyMath.h
- MySnakeModel.h
- MySpline.h
- SnakeHCICommon.h
- stdafx.h
- targetver.h

源文件

- BasicSnakeModel.cpp
- MyMath.cpp
- MySnakeModel.cpp
- MySpline.cpp
- SnakeHCICommon.cpp
- SnakeModelTest.cpp
- stdafx.cpp

资源文件

- ReadMe.txt

SnakeModelTest.cpp

```
// SnakeModelTest.cpp : 定义控制台应用程序
//

#include "stdafx.h"
#include "BasicSnakeModel.h"
#include "MySnakeModel.h"
#include <opencv2/opencv.hpp>
using namespace cv;

CMySnakeModel SnakeObj;

CBasicSnakeModel BasicSnakeObj;

Mat image;//原图像
Mat Displmg;//显示结果的图像

//////////下面是交互程序//////////
```

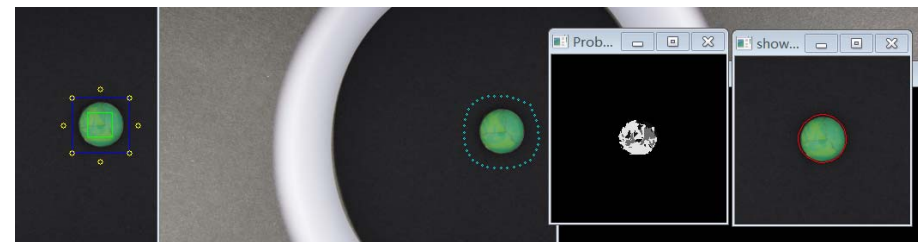
SnakeModelTest.cpp SnakeHCICommon.h MyMath.h MySp

```
{ BFeatureMapStruct
typedef struct BFeatureMapStruct

BFeatureMapStruct

#ifndef _BASICSNAKEMODEL_H_
#define _BASICSNAKEMODEL_H_
#include "MySpline.h"
#include "SnakeHCICommon.h"
#include <opencv2/opencv.hpp>
using namespace cv;

//特征图的一个结构体
typedef struct BFeatureMapStruct
{
    Mat eline;//这个外力, 即图像的强度
    Mat eedge;//梯度图像
    Mat eterm;//曲率图像
    ...
}
```



一个蛇模型算法框架实例

2. 遍历图像上所有像素（例如把图像左右翻转）

访问图像中像素的几种方法

1. 使用Mat_模板子类访问图像元素

```
Mat_<uchar> img=Mat::zeros(cvSize(300,200),CV_8U);
for(int j=0;j<img.rows/2;j++)
{
    for(int i=0;i<img.cols/2;i++)
    {
        img(j,i)=255;
    }
}
cout<<"//函数功能：使用Mat_模板子类更简洁访问图";
imshow("show",img);
cvWaitKey(0);
```

2. 以.at<>方式访问

```
cout<<"//函数功能：给图片添加椒盐噪声"<<endl;
for(int k=0;k<n;k++)
{
    int i=rand()%image.cols;
    int j=rand()%image.rows;
    if(image.channels()==1)//灰度图
    {
        image.at<uchar>(j,i)=255;
    }
    else if(image.channels()==3)//彩色图
    {
        image.at<Vec3b>(j,i)[0]=255;
        image.at<Vec3b>(j,i)[1]=255;
        image.at<Vec3b>(j,i)[2]=255;
    }
}
```

3. 通过访问图像每行以及以in-place的方式访问

```
int nl=image.rows;//行数
//每行的元素个数
int nc=image.cols*image.channels();
int k=0;
cout<<"//函数功能：通过访问图像每行";
for(int j=0;j<nl;j++)
{
    //得到第j行的首地址
    uchar* data=image.ptr<uchar>(j);
    for(int i=0;i<nc;i++)
    {
        if(k==0)
        {
            int tt=data[i];
            cout<<tt<<endl;
        }
        //处理每一个像素
        //data[i]=data[i]/div*div+div/2;//慢速,
        *data++=*data/div*div+div/2;//快速
    }
    if(k==0)
    {
        int tt=data[i];
        cout<<tt<<endl;
        k++;
    }
}
```

4. 通过迭代器方式访问

```
//得到初始位置的迭代器
Mat_<Vec3b>::iterator it=image.begin<Vec3b>();
//得到终止位置的迭代器
Mat_<Vec3b>::iterator itend=image.end<Vec3b>();
cout<<"//函数功能：通过访问图像每行，以迭代器";
//遍历所有像素
for(;it!=itend;++it)
{
    //处理每个像素
    (*it)[0]=(*it)[0]/div*div+div/2;
    (*it)[1]=(*it)[1]/div*div+div/2;
    (*it)[2]=(*it)[2]/div*div+div/2;
}
```

5. 通过通过三行指针联合访问

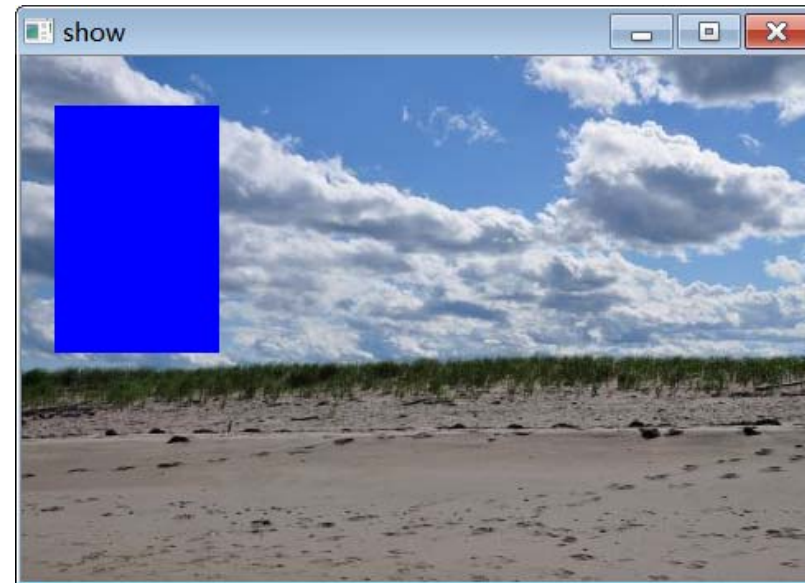
```
//如有必要则分配图像
cout<<"//函数功能：锐化图像-通过三行指针"<<endl;
result.create(img.size(),img.type());
for(int j=1;j<img.rows-1;j++)//处理除了第一行和最后一行之外的所有行
{
    const uchar* previous=img.ptr<const uchar>(j-1);//上一行
    const uchar* current=img.ptr<const uchar>(j);//当前行
    const uchar* next=img.ptr<const uchar>(j+1);//下一行
    uchar* output=result.ptr<uchar>(j);//输出行

    for(int i=1;i<img.cols-1;i++)
    {
        *output++=saturate_cast<uchar>(5*current[i]-current[i-1]-current[i+1]-previous[i]-next[i]);
    }
}
//将未处理的像素设置为0
result.row(0).setTo(Scalar(0));
result.row(result.rows-1).setTo(Scalar(0));
result.col(0).setTo(Scalar(0));
result.col(result.cols-1).setTo(Scalar(0));
```

3. 操纵图像局部感兴趣区域

```
cout<<"函数功能： 叠加图像到ROI区域"<<endl;
Mat image=imread("beach.jpg",1);
Mat ImgROI=image (Rect(20,30,100,150)); //w,h
Mat Mask (150,100,CV_8UC3,Scalar(255,0,0)); //h,w
//Mat Mask=Mat::zeros(150,100,CV_8UC3); //h,w

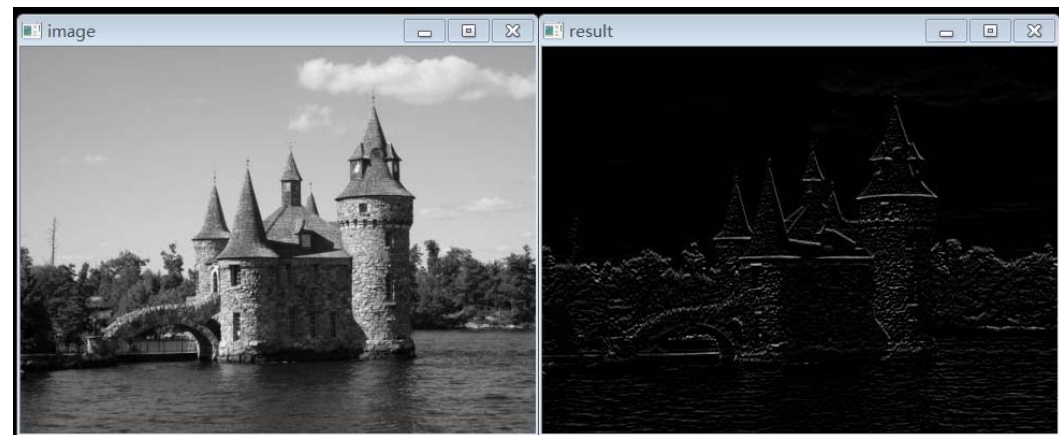
//addWeighted(ImgROI,1,Mask,0.3,0.,ImgROI);
Mask.copyTo(ImgROI);
imshow("show",image);
cvWaitKey(0);
```



4. 滤波算子对图像进行滤波

```
cout<<"函数功能: OpenCV中的Filter2D函数"<<endl;
Mat image=imread("boldt.jpg",0);
//Mat grayimg;
//cvtColor(image,grayimg,CV_RGB2GRAY,0);
Mat kernel (3,3,CV_32F,Scalar(0));
kernel.at<float>(0,1)=1;//Sobel
//kernel.at<float>(1,0)=-1;
kernel.at<float>(2,1)=-1;
//kernel.at<float>(1,2)=-1;
kernel.at<float>(1,1)=0;
Mat result;
filter2D(image,result,CV_8U,kernel,Point(-1,-1));
imshow("image",image);
imshow("result",result);
cvWaitKey(0);
```

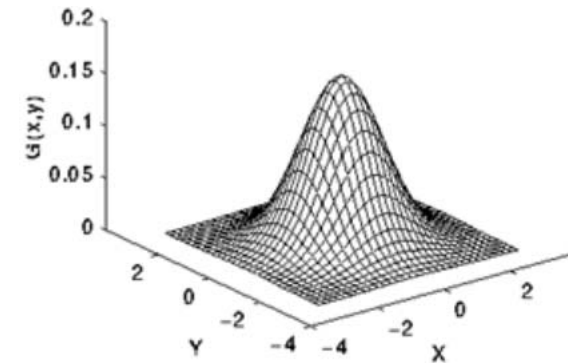
0	1	0
0	0	0
0	-1	0



4. 滤波算子对图像进行滤波

-1	0	1
-1	0	1
-1	0	1

0	1	0
1	-4	1
0	1	0



1	1	1
0	0	0
-1	-1	-1

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

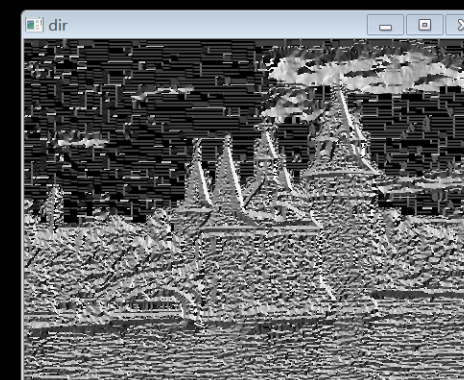
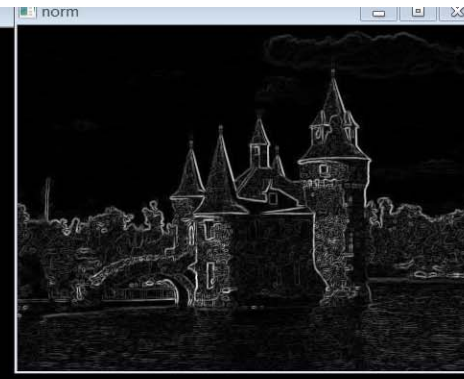
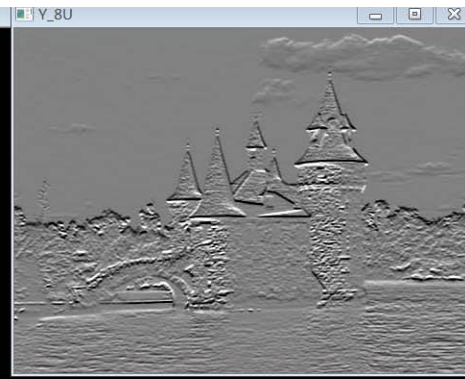
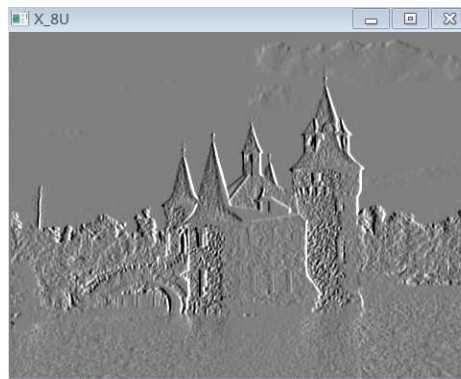
$\frac{1}{273}$

Sobel算子

锐化算子

高斯滤波算子

4. 滤波算子对图像进行滤波



4. 滤波算子对图像进行滤波



-1	0	1
-1	0	1
-1	0	1

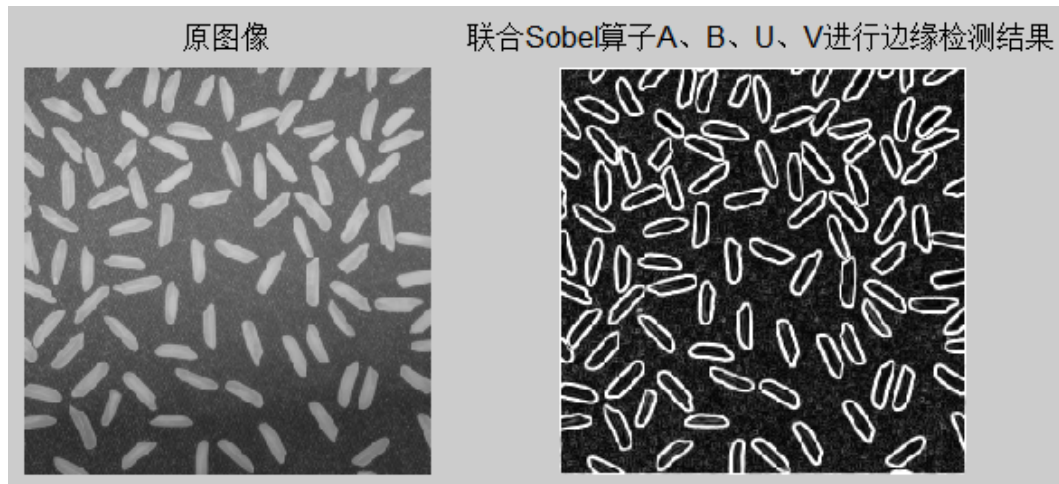
1	1	1
0	0	0
-1	-1	-1



4.滤波算子对图像进行滤波



5.最优的梯度幅值图像



四个方向Sobel算子

$$A = \begin{bmatrix} -1 & -2 & -1; \\ 0 & 0 & 0; \\ 1 & 2 & 1 \end{bmatrix};$$

$$B = \begin{bmatrix} -1 & 0 & 1; \\ -2 & 0 & 2; \\ -1 & 0 & 1 \end{bmatrix};$$

$$U = \begin{bmatrix} -2 & -1 & 0; \\ -1 & 0 & 1; \\ 0 & 1 & 2 \end{bmatrix};$$

$$V = \begin{bmatrix} 0 & 1 & 2; \\ -1 & 0 & 1; \\ -2 & -1 & 0 \end{bmatrix};$$

5.最优的梯度幅值图像

```

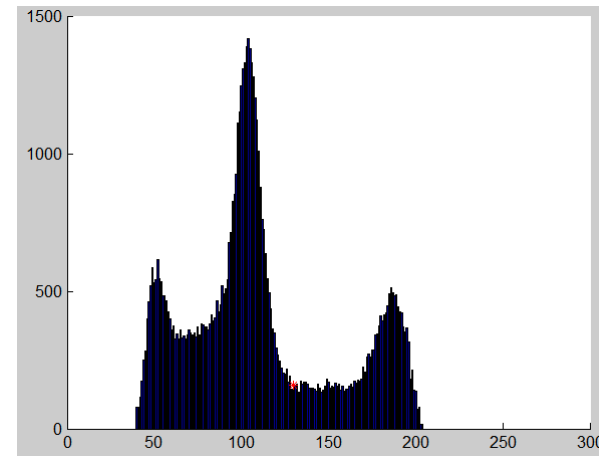
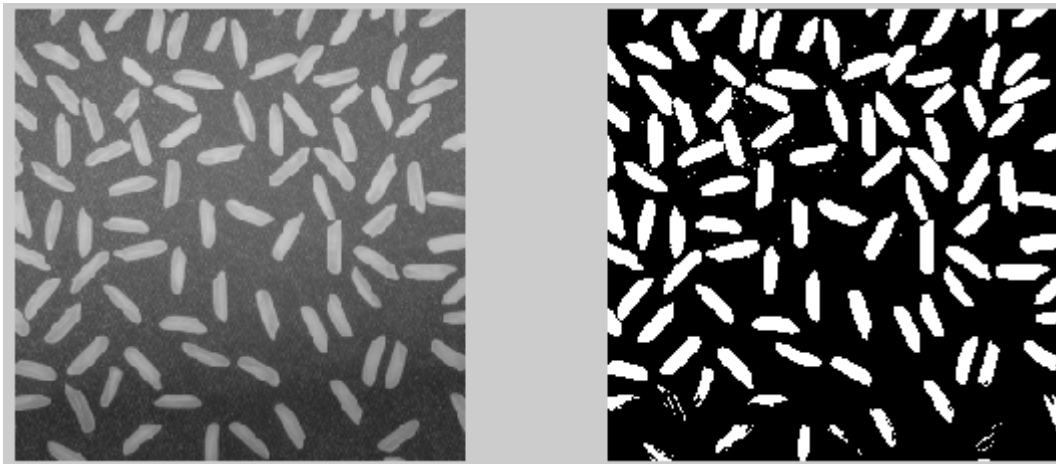
1  % 联合Sobel算子A、B、U、V进行边缘检测结果
2  close all;clear;clc;
3  I=imread('rice.png');%m*n的单通道灰度图
4  subplot(1,2,1)
5  imshow(I)
6  title('原图像');
7  [h,w]=size(I);
8  I_reshape=zeros(h+2,w+2);%创建一个上下左右均多一个像素宽的零矩阵
9  I_reshape(2:h+1,2:w+1)=I;%把I矩阵拷贝到新创建矩阵中间
10 g=zeros(h,w);%用于保存结果的矩阵
11 A=[-1 -2 -1;
12     0 0 0;
13     1 2 1];
14 B=[-1 0 1;
15     -2 0 2;
16     -1 0 1];
17 U=[-2 -1 0;
18     -1 0 1;
19     0 1 2];
20 V=[0 1 2;
21     -1 0 1;
22     -2 -1 0];
23 %I在I_reshape矩阵中纵向方向的范围是j=2到j=h+1,其中j=1和j=h+2对应0像素值
24 %I在I_reshape矩阵中横向方向的范围是i=2到i=w+1,其中i=1和i=w+2对应0像素值
25 for j=2:h+1
26     for i=2:w+1
27         aa=0;
28         bb=0;
29         uu=0;
30         vv=0;
31         for r=-1:1
32             for s=-1:1
33                 aa=aa+I_reshape(j+r,i+s)*A(r+2,s+2);%在3*3领域进行卷积操作
34                 bb=bb+I_reshape(j+r,i+s)*B(r+2,s+2);%在3*3领域进行卷积操作
35                 uu=uu+I_reshape(j+r,i+s)*U(r+2,s+2);%在3*3领域进行卷积操作
36                 vv=vv+I_reshape(j+r,i+s)*V(r+2,s+2);%在3*3领域进行卷积操作
37             end
38         end
39         t=max(abs(aa),abs(bb));
40         t=max(t,abs(uu));
41         t=max(t,abs(vv));
42         g(j-1,i-1)=t;%取最大值作为模值
43     end
44 end
45 subplot(1,2,2)
46 g=uint8(g);
47 imshow(g)
48 title('联合Sobel算子A、B、U、V进行边缘检测结果');

```

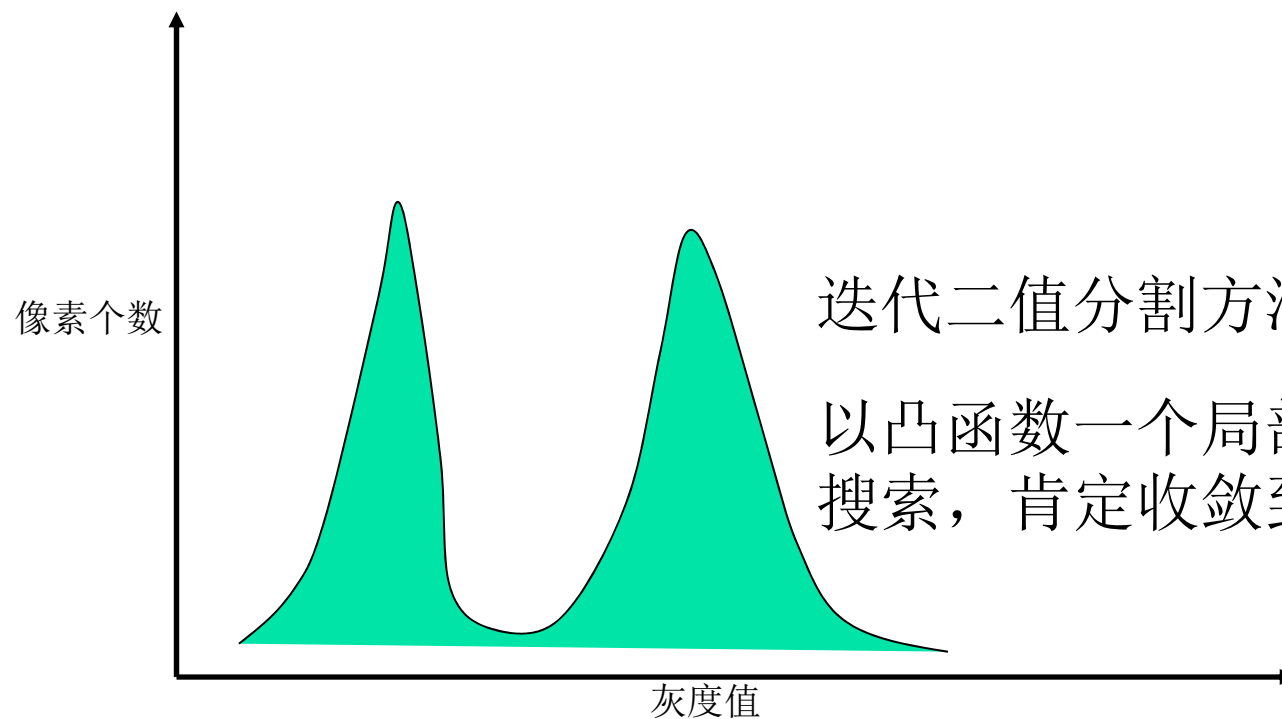
根据算法思想，把程序
转换为VC+OpenCV形式
或Python+OpenCV形式

6.学习对图像进行二值化的方法（动态阈值法）

采用迭代法可以自适应寻找灰度图像中的最佳分割阈值，迭代法的基本原理是首先找出图像的最大灰度值和最小灰度值，然后取平均值作为初始的分割阈值 T ，那么 T 可以把图像分割为两部分，即灰度值大于 T 的部分和灰度值小于 T 的部分；分别计算这两部分的平均灰度值 H 和 L ，更新 $T=(H+L)/2$ ；重复以上步骤直至算法收敛。编写迭代法求最佳阈值的代码。



6.学习对图像进行二值化的方法（动态阈值法）



迭代二值分割方法的假设

以凸函数一个局部最优解开始
搜索，肯定收敛到全局最优解。

STL模板库

Vector中的删除操作

```
#include <list>
#include <vector>
using namespace std;

int main(int argc, char* argv[])
{
    //删除指定vector元素
    vector<int> intervectorlist;
    for(int i=0; i<10; i++)
    {
        intervectorlist.push_back(i);
    }
    vector<int>::iterator it_vector;
    for(it_vector=intervectorlist.begin(); it_vector!=intervectorlist.end(); )
    {
        if(*it_vector==2 || *it_vector==7)
        {
            printf("Delete=%d\n", (*it_vector));
            it_vector=intervectorlist.erase(it_vector); //删除元素，返回值指向已删除元素的下一个位置
            printf("删除结束\n");
        }
        else
        {
            it_vector++;
        }
    }
    //输出
    printf("删除vector\n");
    for(int i=0; i<intervectorlist.size(); i++)
    {
        printf("%d\n", intervectorlist[i]);
    }
    printf("删除全部\n");
    vector<int>().swap(intervectorlist);
    for(int i=0; i<intervectorlist.size(); i++)
    {
        printf("%d\n", intervectorlist[i]);
    }

    printf("\n");
}
```

List中的删除操作

```
//删除指定list元素
list<int> interlist;
for(int i=0; i<10; i++)
{
    interlist.push_back(i);
}
list<int>::iterator it;
for(it=interlist.begin(); it!=interlist.end(); )
{
    if(*it==2 || *it==7)
    {
        printf("Delete=%d\n", (*it));
        interlist.erase(it++); //删除元素，返回值指向已删除元素的下一个位置
        printf("删除结束\n");
    }
    else
    {
        it++;
    }
}
printf("删除全部\n");
if(interlist.size()>0)
{
    list<int>::iterator it;
    for(it=interlist.begin(); it!=interlist.end(); )
    {
        it=interlist.erase(it++);
    }
}
```

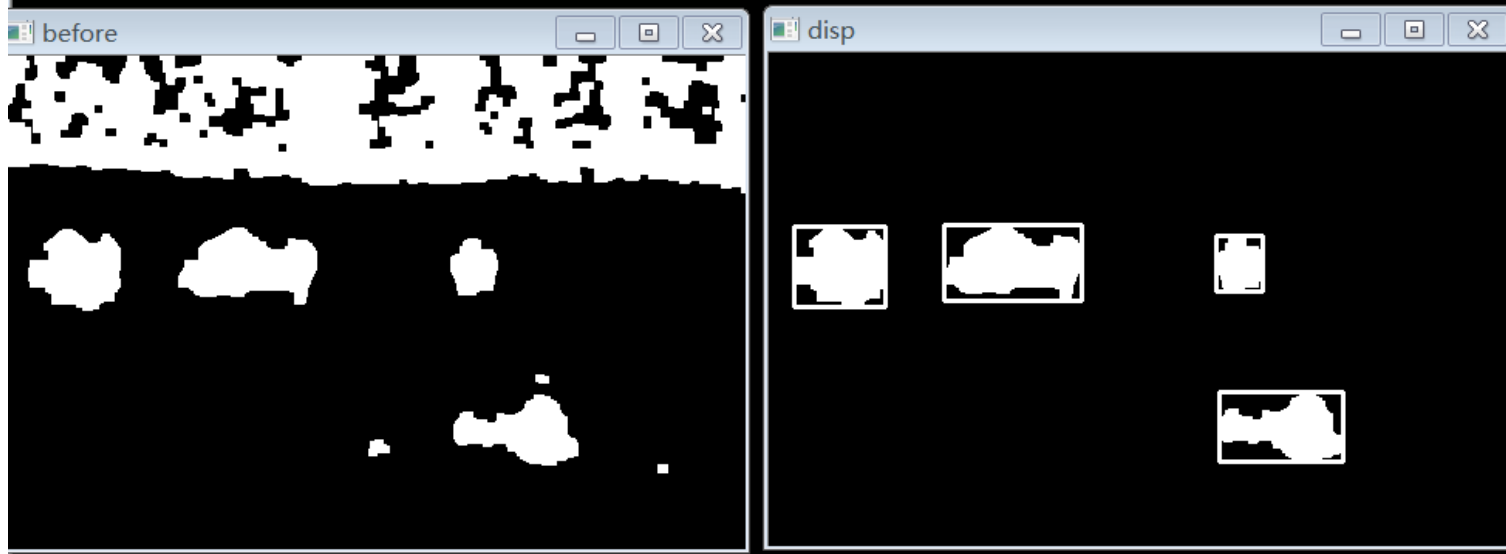
7. 获取图像中的轮廓，对图像中的目标进行计数



7. 获取图像中的轮廓，对图像中的目标进行计数

函数功能：求轮廓图，然后显示大小大于某阈值的那些图

```
i=1, size=42  
i=2, size=154  
i=3, size=3726  
i=4, size=54  
i=5, size=1147  
i=6, size=3180  
i=7, size=4500  
i=8, size=108  
i=9, size=42064
```



```
void CForFreshMen::ContourFunSomeSizeShow()
{
    cout<<"函数功能：求轮廓图，然后显示大小大于某阈值的那些图"<<endl;
    Mat image=imread("binaryGroup.bmp",0);
    Mat ShowImg (image.size(),CV_8U,Scalar(0));

    vector<vector<Point>>>contours;
    imshow("before",image);
    findContours(image, //注意，这个函数会修改image图像的内容
        contours, // a vector of contours
        CV_RETR_EXTERNAL, //retrieve the external contours 或者： CV_RETR_LIST //retrieve all contours
        CV_CHAIN_APPROX_NONE); // all pixels of each contours

    vector<vector<Point>>>contours_tmp;
    float T=1000;//矩形面积
    for(int i=0;i<contours.size();i++)
    {
        vector<Point> Points=contours[i];//如何获取一个包围框的点集
        Rect rect=boundingRect(Points);//如何获取该点集的外围框
        printf("i=%d, size=%d\n",i+1,rect.width*rect.height);
        if(rect.width*rect.height>T&&rect.width*rect.height<5000)
        {
            //存储
            contours_tmp.push_back(Points);

            //显示
            //显示指定的轮廓
            //方法1： 使用离散点的方式画出目标的轮廓
            int i;
            for(i=0;i<Points.size()-1;i++)
            {
                line(ShowImg,Points[i],Points[i+1],Scalar(255),1,8,0);
            }
            line(ShowImg,Points[i],Points[Points.size()-1],Scalar(255),1,8,0);

            //方法2： 可以按照以下格式，逐个填充轮廓
            vector<vector<Point>>>M;
            M.push_back(Points);
            fillPoly(ShowImg,M,Scalar(255));//画出填充结果

            //方法3：
            Rect b=boundingRect(Points);//如何获取该点集的外围框
            rectangle(ShowImg,b,Scalar(255,255,255),2);//画出框框
        }
    }
    imshow("disp",ShowImg);
    cvWaitKey(0);
}
```

上机内容:

- 配置好基于Python+OpenCV的图像处理环境，或基于Visual Studio 2015/2017+OpenCV的图像处理环境
- 课堂实验：编程实现本次课程提到的7个编程任务，包括图像显示、图像遍历、感兴趣区域操纵、滤波算法（包括基于Sobel算子、锐化算子与高斯算子的边缘检测程序）、最优梯度幅值边缘检测方法、图像二值化方法及对图像中目标轮廓的操纵